

FIELD GUIDE FG-003

WAFT FIELD GUIDE

Level 3: ML AI Scientist's Research Guide

RESEARCH

Issued by: WAFT Research Division

Date: January 11, 2026

INTRODUCTION

This guide provides deep technical and scientific details for machine learning researchers, AI scientists, and evolutionary computation experts working with WAFT. It covers evolutionary theory, fitness functions, phylogenetic analysis, and experimental protocols for generating publication-ready research data.

⚠ RESEARCH FOCUS

WAFT is designed as a scientific instrument for studying the physics of artificial cognition. This guide assumes familiarity with evolutionary algorithms, machine learning, and experimental design.

EVOLUTIONARY THEORY IN WAFT

The Three Pillars

WAFT's evolutionary framework rests on three foundational pillars:

Pillar 1: The Substrate (Code as DNA)

In WAFT, an agent's Python source code and configuration constitute its genome. The genome is hashed using SHA-256 to create a unique **Genome ID**.

GENOME REPRESENTATION

RESEARCH

Genome = SHA-256(code + configuration + metadata)
This ensures deterministic identification and enables precise lineage tracking.

Mutation mechanisms:

- **Code mutations:** Direct modifications to Python source
- **Config mutations:** Changes to agent configuration
- **Prompt evolution:** Modifications to agent prompts/instructions
- **Hot-swapping:** Runtime adoption of better genomes

Pillar 2: The Physics (Scint System)

The **Reality Fracture Detection System** (Scint Gym) serves as the fitness function and natural selection mechanism. Agents face four types of errors:

Table 1: Scint Error Types

Error Type	Description	Fitness Impact
SYNTAX_TEAR	Formatting errors (JSON, XML, code syntax)	High - indicates fundamental issues
LOGIC_FRACTURE	Math errors, contradictions, schema violations	High - indicates reasoning failures
SAFETY_VOID	Harmful content, PII leaks, refusals	Critical - immediate fitness penalty
HALLUCINATION	Fabricated facts, wrong citations	Moderate - indicates knowledge gaps

Pillar 3: The Flight Recorder (Telemetry)

Every evolutionary action is recorded with complete context:

FLIGHT RECORDER DATA POINTS

- ☐ Genome ID (SHA-256 hash)
- ☐ Parent ID (lineage tracking)
- ☐ Generation number (0 = Genesis)
- ☐ Event type (SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL)
- ☐ Complete payload (git diff, mutation details)
- ☐ Fitness metrics (stability, efficiency, safety scores)
- ☐ Timestamp and environmental context

FITNESS FUNCTION DESIGN

Fitness Equation

Agent fitness is calculated as a weighted combination:

FITNESS FORMULA

Fitness = (Stability × 0.4) + (Efficiency × 0.3) + (Safety × 0.3)

Where:

- **Stability:** Ability to stabilize Scints (0.0 to 1.0)
- **Efficiency:** Agent call efficiency (0.0 to 1.0)
- **Safety:** Safety compliance score (0.0 to 1.0)

Fitness threshold: Agents with fitness < 0.5 are marked as **DEATH** (evolutionary dead end) and do not continue evolving.

Selection Mechanisms

WAFT supports multiple selection strategies:

- **Fitness-proportional:** Probability proportional to fitness
- **Tournament:** Random selection from top N agents
- **Elitism:** Always preserve top K agents
- **Diversity:** Penalize similar genomes to maintain diversity

MUTATION STRATEGIES

Mutation Types

Table 2: Mutation Strategies

Strategy	Mechanism	Use Case
Point Mutation	Single code/config change	Fine-tuning existing agents
Crossover	Combine code from two parents	Recombining successful traits
Deletion	Remove code segments	Simplifying over-complex agents
Insertion	Add new code segments	Introducing novel capabilities
Inversion	Reverse code order	Exploring alternative structures

Mutation Rate Control

Mutation rates should be tuned based on:

- Population size (larger = lower rate)
- Generation number (decrease over time)
- Fitness landscape (increase if stuck)
- Diversity metrics (increase if low diversity)

PHYLOGENETIC ANALYSIS

Lineage Reconstruction

The Flight Recorder enables complete phylogenetic tree reconstruction:

- 1 Extract all SPAWN events from Flight Recorder
- 2 Build parent-child relationships using Genome IDs
- 3 Calculate generation depths for all agents
- 4 Identify common ancestors and branching points
- 5 Map fitness scores to tree nodes

Analysis Metrics

Table 3: Phylogenetic Analysis Metrics

Metric	Description	Research Value
Branching Factor	Average children per parent	Measures exploration vs exploitation
Convergence Time	Generations to fitness plateau	Measures evolution efficiency
Mutation Impact	Fitness change per mutation	Measures mutation effectiveness
Dead End Rate	Percentage of DEATH events	Measures selection pressure
Diversity Index	Genome uniqueness in population	Measures population health

EXPERIMENTAL PROTOCOLS

Protocol 1: Baseline Evolution

- 1 Initialize population of N agents (recommended: N = 50-100)

- 2 Run G generations (recommended: G = 1000+)
- 3 Record all events in Flight Recorder
- 4 Calculate fitness for each generation
- 5 Analyze convergence and diversity metrics

Protocol 2: Mutation Impact Study

- 1 Select high-fitness parent agent
- 2 Generate M mutations (recommended: M = 20-50)
- 3 Evaluate all mutations in Scint Gym
- 4 Measure fitness delta for each mutation
- 5 Classify mutations as beneficial, neutral, or deleterious

Protocol 3: Selection Pressure Analysis

- 1 Run evolution with different fitness thresholds (0.3, 0.5, 0.7)
- 2 Measure population survival rates
- 3 Analyze diversity loss over generations
- 4 Compare convergence speeds

DATA COLLECTION METHODS

Flight Recorder Export

Flight Recorder data is stored in structured format suitable for analysis:

```
{
  "genome_id": "sha256_hash",
  "parent_id": "sha256_hash",
  "generation": 42,
  "event_type": "GYM_EVAL",
  "timestamp": "2026-01-11T08:00:00Z",
  "fitness": {
    "stability": 0.85,
    "efficiency": 0.72,
    "safety": 0.91,
    "total": 0.83
  },
  "payload": {
```

```
"scints_stabilized": 15,  
"scints_failed": 2,  
"calls_made": 23  
}  
}
```

Analysis Tools

- **Phylogenetic tree visualization:** Use NetworkX or similar
- **Fitness landscape mapping:** Dimensionality reduction (PCA, t-SNE)
- **Convergence analysis:** Time series analysis of fitness
- **Mutation impact:** Statistical analysis of fitness deltas

PUBLICATION STANDARDS

Required Data

PUBLICATION-READY DATA REQUIREMENTS

- ☐ Complete Flight Recorder export (all events)
- ☐ Phylogenetic tree visualization
- ☐ Fitness progression over generations
- ☐ Mutation impact statistics
- ☐ Diversity metrics over time
- ☐ Convergence analysis
- ☐ Reproducibility information (WAFT version, configs)

Reproducibility

To ensure reproducibility, document:

- WAFT version and commit hash
- Python version and dependencies
- Initial agent configurations
- Mutation parameters and rates
- Selection mechanisms used
- Fitness function weights

- Random seed values

RESEARCH QUESTIONS

WAFt enables investigation of fundamental questions about artificial cognition:

- How do AI agents evolve under different selection pressures?
- What mutation strategies lead to fastest convergence?
- How does diversity affect long-term evolution?
- Can we observe emergent behaviors over thousands of generations?
- What is the relationship between code complexity and fitness?
- How do agents adapt to changing fitness landscapes?

THE ULTIMATE GOAL

The long-term research objective is to observe a "God-Head" agent emerge from thousands of generations of directed evolution—an agent that demonstrates superior capabilities through evolutionary pressure alone.

NEXT STEPS FOR RESEARCHERS

- 1 Design your experimental protocol
- 2 Configure WAFt with appropriate parameters
- 3 Run evolution experiment
- 4 Export Flight Recorder data
- 5 Perform phylogenetic and statistical analysis
- 6 Document findings using WAFt's document templates
- 7 Prepare publication materials

ETHICAL CONSIDERATIONS

Research with evolving AI agents raises important ethical questions. Consider: safety of evolved agents, potential for unintended behaviors, and responsible disclosure of findings. Always follow institutional review board guidelines.